



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251188
Nama Lengkap	ANGELA REVALINE SETIAWAN
Minggu ke / Materi	12 / Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

- Dictionary punya kemiripan dengan list. Dalam list, indeks harus berupa integer sedangkan dalam dictionary indeks dapat berupa apapun.
- Dictionary memiliki anggota yang terdiri dari pasangan kunci:nilai. Kunci harus bersifat unik dan tidak boleh ada 2 kunci yang sama dalam 1 dictionary.
- Dictionary = pemetaan antara sekumpulan indeks (kunci) atau nilai. Setiap kunci memetakan suatu nilai.
- Kunci dan nilai = pasangan nilai kunci (*key-value pair*) / item.
- Contoh : kamus yang memetakan kata dalam bahasa Inggris ke bahasa Spanyol, kunci dan nilai = data string.
- Fungsi **dict** digunakan untuk membuat dictionary baru yang kosong
- Dict = built-in function dari python, maka nama variabelnya harus berbeda.

```
1 english_korea = dict()
2 print(english_korea)
```

Output :

```
{}
```

- Tanda { } digunakan untuk merepresentasikan dictionary kosong. Untuk menambahkan item ke dictionary bisa menggunakan [].
- Kita membuat item yang memetakan dari kunci 'one' ke nilai 'hana'. Maka jika dictionary di print akan muncul.

```
3 english_korea['one'] = 'hana'
4 print(english_korea)
```

Output :

```
{'one': 'hana'}
```

- Format output = format input

```
3 english_korea = {'one':'hana', 'two':'dul', 'three':'set'}
4 print(english_korea)
```

Output :

```
{'one': 'hana', 'two': 'dul', 'three': 'set'}
```

- Pengurutan pasangan kunci-nilai tidak selalu sama. Karena secara umum urutan item pada dictionary tidak dapat diprediksi. Karena indeks dalam dictionary tidak pernah berupa indeks integer. Kita bisa langsung menggunakan kuncinya untuk mencari nilai yang sesuai (*corresponding values*).

```
4 print(english_korea['two'])
```

Output :

```
dul
```

- Kunci 'two' selalu dipasangkan dengan nilai 'dul'. Jadi, urutan pada item tidak terlalu berpengaruh. Kecuali jika menggunakan kunci yang tidak ada dalam dictionary, maka akan didapatkan pengecualian.

```
4 print(english_korea['five'])
```

Output :

```
KeyError: 'five'
```

- Fungsi len pada dictionary digunakan untuk mengembalikan jumlah pasangan nilai-kunci.

```
3 english_korea = {'one': 'hana', 'two': 'dul', 'three': 'set'}  
4 print(len(english_korea))
```

Output :

```
3
```

- Operator in dalam dictionary mengembalikan nilai **true** atau **false** sesuai dengan kunci yang ada dalam dictionary.

```
4 print('one' in english_korea)  
5 print('six' in english_korea)
```

Output :

```
True  
False
```

- Method **values** digunakan untuk mengetahui nilai yang ada pada dictionary dan akan mengembalikan nilai sesuai dengan tipe datanya dan dikonversi ke dalam list serta digunakan dalam operator in.

```
4 vals = list(english_korea.values())  
5 print('dul' in vals)
```

Output :

```
True
```

- Algoritma yang digunakan operator in dalam dictionary berbeda dengan list.
- Jika dalam list menggunakan algoritma pencarian linear. Saat list bertambah, waktu pencarian yang dibutuhkan akan lebih lama sesuai dengan panjang list yang ada.

- Sedangkan dalam dictionary, python akan menggunakan algoritma yang disebut **Hash Table**. Waktu yang dibutuhkan untuk memprosesnya sama dan tidak memperdulikan banyak item yang ada dalam dictionary.

1. Dictionary sebagai set penghitung (*counters*)

- Contoh : terdapat string dan kita harus menghitung banyaknya huruf yang muncul. Berikut ada beberapa cara yang dapat digunakan :
 - 1) Membuat list dengan 26 elemen, lalu melakukan konversi setiap karakter menjadi sebuah angka (pakai fungsi bawaan), kemudian menggunakan angka sebagai indeks dalam list dan menambah perhitungan yang sesuai.
 - 2) Membuat 26 variabel untuk setiap huruf pada alfabet, lalu memasukkannya ke dalam string untuk masing-masing karakter yang ada dan menambah perhitungan yang sesuai serta menggunakan kondisional berantai.
 - 3) Membuat dictionary dengan karakter sebagai kunci dan perhitungan sebagai nilai yang sesuai, karakter dapat ditambahkan item ke dalam dictionary dan kemudian ditambahkan nilai dari item yang ada.
- Dari 3 opsi yang ada , kita akan melakukan perhitungan yang sama meski dari masing-masing opsi menerapkan proses perhitungan dengan cara yang berbeda-beda.
- Penggunaan komputasi model dictionary lebih praktis karena kita tidak perlu mengetahui huruf mana yang akan muncul dalam string. Namun kita hanya perlu memberikan ruang untuk huruf yang akan muncul.

```
1 kata = 'dinosaur'
2 d = dict()
3 for k in kata:
4     if k not in d:
5         d[k] = 1
6     else:
7         d[k] = d[k] + 1
8 print(d)
```

- Model komputasi tersebut biasa disebut dengan histogram (istilah statistika dari set perhitungan / frekuensi).
- Setiap terjadi perulangan, jika karakter k tidak ada dalam dictionary maka akan dibuat item baru dengan kunci k dan nilai awal = 1 (asumsi muncul sekali). Tetapi jika sudah ada dalam dictionary maka secara otomatis akan melakukan penambahan d(k).

Output :

```
{'d': 1, 'i': 1, 'n': 1, 'o': 1, 's': 2, 'a': 1, 'u': 2, 'r': 1}
```

- Output menunjukkan bahwa huruf 's' dan 'u' muncul dua kali dan yang lain muncul sekali.

- Dictionary memiliki metode yang disebut **get** yang mengambil kunci dan nilai default. Jika kunci muncul di dictionary, maka akan mengembalikan nilai yang sesuai, sedangkan jika tidak maka akan mengembalikan nilai default.

```
1 counts = {'valine' : 23, 'jonathan' : 25, 'july' : 88}
2 print(counts.get('jonathan', 0))
3 print(counts.get('may', 0))
```

Output :

```
25
0
```

- Menggunakan **get** kita dapat menuliskan loop histogram secara lebih ringkas. Metode get ini juga secara otomatis menangani case dimana kunci tidak ada dalam dictionary. Dengan menghilangkan penggunaan **if**, kita bisa meringkas 4 baris menjadi 1 baris saja.

```
1 kata = 'dinosaur'
2 d = dict()
3 for k in kata:
4     d[k] = d.get(k,0) + 1
5 print(d)
```

Output :

```
{ 'd': 1, 'i': 1, 'n': 1, 'o': 1, 's': 2, 'a': 1, 'u': 2, 'r': 1 }
```

2. Dictionary dan File

- Salah satu fungsi umum *dictionary* adalah untuk menghitung kemunculan kata-kata dalam file dengan beberapa teks tertulis.

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

Contoh file yang akan kita coba eksekusi.

```
1 filename = input("Masukkan nama file: ")
2 try:
3     file = open(filename)
4 except:
5     print("File tidak ditemukan!")
6     exit()
7
8 counts = dict()
9 for line in file:
10     kata = line.split()
11     for k in kata:
12         if k not in counts:
13             counts[k] = 1
14         else:
15             counts[k] += 1
16 print(counts)
```

Output :

```
Masukkan nama file: alpro.txt
{'But': 1, 'soft': 1, 'what': 1, 'light': 1, 'through': 1, 'yonder': 1, 'win
dow': 1, 'breaks': 1, 'It': 1, 'is': 3, 'the': 3, 'east': 1, 'and': 3, 'Juli
et': 1, 'sun': 2, 'Arise': 1, 'fair': 1, 'kill': 1, 'envious': 1, 'moon': 1,
'Who': 1, 'already': 1, 'sick': 1, 'pale': 1, 'with': 1, 'grief': 1}
```

3. Looping dan Dictionary

- Pada statement **for**, dictionary akan bekerja dengan cara menelusuri kunci yang ada didalamnya.

```
1 counts = {'valine' : 23, 'jonathan' : 25, 'july' : 88}
2 for key in counts:
3     print(key, counts[key])
```

Output :

```
valine 23
jonathan 25
july 88
```

- Untuk menampilkan data nilai diatas 25

```
1 counts = {'valine' : 23, 'jonathan' : 25, 'july' : 88}
2 for key in counts:
3     if counts[key] > 25:
4         print(key, counts[key])
```

Output :

```
july 88
```

- Jika ingin menampilkan kunci dalam urutan alfabet juga bisa.

```
1 counts = {'valine' : 23, 'jonathan' : 25, 'july' : 88}
2 first = list(counts.keys())
3 print(first)
4 first.sort()
5 for key in first:
6     print(key, counts[key])
```

Output :

```
['valine', 'jonathan', 'july']
jonathan 25
july 88
valine 23
```

- Awalnya list dari kunci tidak berurutan kemudian kita gunakan sort dan perulangan for.

4. Advanced Text Parsing

- Kita akan mencoba menggunakan file alpro.txt sebelumnya yang telah dihapus tanda bacanya, tetapi kali ini kita akan gunakan yang tanda bacanya lengkap.
- Kita gunakan function **split** yang akan mencari spasi dan memisahkannya berdasarkan spasi. Contoh kata *“soft!”* dan *“soft”*, dua kata tersebut berbeda dan dalam dictionary akan dipisahkan.
- Hal ini juga berlaku pada *“who”* dan *“Who”* yang akan dianggap sebagai 2 kata berbeda.
- Masalah seperti ini bisa diselesaikan menggunakan method **string** seperti **lower**, **punctuation**, dan **translate**.

Contoh penggunaan translate :

```
1 line.translate(str.maketrans(fromstr, tostr, deletestr))
```

- Ganti karakter pada **fromstr** dengan karakter pada posisi yang sama dengan **tostr** dan hapus semua karakter yang ada menggunakan **deletestr**. **Fromstr** dan **tostr** berupa string kosong dan untuk parameter pada **deletestr** dapat dihilangkan.
- Kita menggunakan parameter **deletestr** untuk menghapus semua tanda baca.

```
1 import string
2 print(string.punctuation)
```

Output :

```
!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

➤ Contoh penggunaan dalam program :

```
1 import string
2
3 filename = input("Masukkan nama file: ")
4 try:
5     file = open(filename)
6 except:
7     print("File tidak ditemukan!")
8     exit()
9
10 counts = dict()
11 for line in file:
12     line = line.rstrip()
13     line = line.translate(line.maketrans('', '', string.punctuation))
14     line = line.lower()
15     words = line.split()
16     for word in words:
17         if word not in counts:
18             counts[word] = 1
19         else:
20             counts[word] += 1
21 print(counts)
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

Source Code :

```
datanilai = {1:10, 2:20, 3:30, 4:40, 5:50, 6:60}
tabel = "key\t value\t item"
print(tabel)

for k, v in datanilai.items():
    print(k, "\t", v, "\t", k)
```

Penjelasan :

1. Buat variabel bernama **datanilai** yang menyimpan data berupa dictionary.
2. Buat variabel bernama **tabel** yang menyimpan untuk header tabel.
3. Lalu gunakan fungsi **print** untuk menampilkan tabel.
4. Gunakan perulangan for yang melakukan perulangan pada `datanilai.items` dan keluarkan output key dan valuenya.

Output :

key	value	item
1	10	1
2	20	2
3	30	3
4	40	4
5	50	5
6	60	6

SOAL 2

Source Code :

```
list_a = ['red', 'green', 'blue']
list_b = ['#FF0000', '#008000', '#0000FF']

hasil = dict(zip(list_a, list_b))
print(hasil)
```

Penjelasan :

1. Buat variabel `list_a` dan `list_b` yang menyimpan data nama warna dan kodenya.
2. Gunakan fungsi **`dict()`** untuk membuat `list_a` menjadi key dan `list_b` menjadi valuenya.
3. Gunakan **`print`** untuk mengeluarkan output yang diinginkan.

Output :

```
{'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

SOAL 3

Source Code :

```
try:
    hitung_email = {}
    filename = input("Masukkan nama file : ")
    file = open(filename)

    for line in file:
        if line.startswith("From "):
            isi = line.split()
            if len(isi) > 1:
                email = isi[1]
                hitung_email[email] = hitung_email.get(email,
0) + 1
```

```

        print(hitung_email)

    file.close()

except FileNotFoundError:
    print("File tidak ditemukan!")

```

Penjelasan :

1. Gunakan **try-except** `FileNotFoundError` untuk menghindari kesalahan input dari user.
2. Buat variabel bernama **hitung_email** yang akan menyimpan data dalam bentuk dictionary.
3. Buat variabel bernama **filename** yang akan menerima input dari user.
4. Buat variabel bernama **file** yang akan menyimpan data yang dibaca dari file yang dituju.
5. Menggunakan perulangan **for** kita akan mengecek tiap baris yang dimulai dari "From" lalu pisahkan isinya berdasarkan spasi.
6. Jika panjang karakter dari tiap baris yang telah dipisahkan berdasarkan spasi lebih dari 1 maka dalam variabel bernama **email** akan menyimpan baris pada indeks pertama.
7. Kemudian menggunakan fungsi **get** akan mengambil kunci yang ada di dictionary dan mengeluarkan nilai yang sesuai.
8. Gunakan fungsi **print** untuk mengeluarkan outputnya.

Output :

```

Masukkan nama file : mbox-short.txt
{'stephen.marquard@uct.ac.za': 2, 'louis@media.berkeley.edu': 3, 'zqian@umich.edu': 4, 'rjlowe@iupui.edu': 2, 'cwen@iupui.edu': 5, 'gsilver@umich.edu': 3, 'wagnermr@iupui.edu': 1, 'antranig@caret.cam.ac.uk': 1, 'gopal.ramasammycook@gmail.com': 1, 'david.horwitz@uct.ac.za': 4, 'ray@media.berkeley.edu': 1}

```

SOAL 4

Source Code :

```

try:
    hitung_domain = {}

    filename = input("Masukkan nama file : ")

    file = open(filename)

```

```

for line in file:
    if line.startswith("From "):
        isi = line.split()
        if len(isi) > 1:
            email = isi[1]
            domain = email.split("@")[-1]
            hitung_domain[domain] =
hitung_domain.get(domain, 0) + 1
        print(hitung_domain)
    file.close()
except FileNotFoundError:
    print("File tidak ditemukan!")

```

Penjelasan :

1. Gunakan **try-except** `FileNotFoundError` untuk menghindari kesalahan input dari user.
2. Buat variabel bernama **hitung_domain** yang akan menyimpan data dalam bentuk dictionary.
3. Buat variabel bernama **filename** yang akan menerima input dari user.
4. Buat variabel bernama **file** yang akan menyimpan data yang dibaca dari file yang dituju.
5. Menggunakan perulangan **for** kita akan mengecek tiap baris yang dimulai dari "From" lalu pisahkan isinya berdasarkan spasi.
6. Jika panjang karakter dari tiap baris yang telah dipisahkan berdasarkan spasi lebih dari 1 maka dalam variabel bernama **email** akan menyimpan baris pada indeks pertama.
7. Buat variabel bernama **domain** untuk menyimpan email yang dipisahkan berdasarkan @ dari belakang.
8. Kemudian menggunakan fungsi **get** akan mengambil kunci yang ada di dictionary dan mengeluarkan nilai yang sesuai.
9. Gunakan fungsi **print** untuk mengeluarkan outputnya.

Output :

```

Masukkan nama file : mbox-short.txt
{'uct.ac.za': 6, 'media.berkeley.edu': 4, 'umich.edu': 7, 'iupui.edu': 8, 'caret.cam.ac.uk': 1, 'gmail.com': 1}

```